INVENTRIX

A Multi-Tenant Smart Retail Store Management & B2B Supplier Integration Platform

ABSTRACT

Inventrix is a modern, unified software solution designed to solve the critical challenges of store branches management, point-of-sale cashier checkouts, and wholesale supplier collaboration. The platform integrates real-time barcode scanning capabilities to speed up retail line checkouts and tracks sales analytics locally in Indian Standard Time (IST). When items are depleted, low stock notifications are pushed via WebSockets to cashier clients and store owners immediately. The application automatically generates customized PDF invoices with direct Rupee currency (₹) rendering and transmits them to customers over secure email relays.

Project Implementation Specification Document
Software Release: v1.2.0-LTS
Environment: Development & Staging

2. Table of Contents

3. Project Overview

3.1 Purpose & Objectives

The objective of Inventrix is to deliver a highly responsive, unified management console that integrates retail physical checkouts (POS) with business operations (employee shifts, inventory thresholds, and multi-tenant store structures) and supplier coordination. Unlike traditional isolated checkout databases, Inventrix enables seamless information flow: automated low-stock warnings instantly notify operators, real-time scanning populates sales carts, and timezones are adjusted to native Indian Standard Time (IST) on aggregates so business owners receive logical, actionable analytics reports.

3.2 Target Users & Business Use Cases

- **Store Owners / Managers:** Analyze daily operations, compare profit margins, deploy cashier custom roles, check purchase order statuses, and review low-stock signals across multiple retail branches.
- **Cashiers / Counter Staff:** Process sales transactions using integrated barcode scanner inputs, choose payment methods, log schedules, and view alerts directly in the header notification panel.
- **Wholesale Suppliers:** Verify purchase orders from stores, accept requests with custom delivery timelines, reject invalid order items, and list wholesale products.
- **Retail Customers:** Check product stock availability, prices, and locations transparently through a dedicated portal.

3.3 Tech Stack Specification

- **Frontend Framework:** React (bootstrapped with Vite) utilizing native State/Effect hooks for client-side data binding and routing.
- **Backend Framework:** Node.js environment with Express for structuring RESTful controller pathways.
- **Database Engine:** MongoDB (NoSQL) with Mongoose ODM for establishing document structures, aggregations, and associations.
- **WebSocket Protocol:** Socket.io (Client & Server libraries) for bidirectional live scanner events and push alerts.
- **Utility Frameworks:** PDFKit (custom server-side PDF generator with local Calibri font assets for unicode support) and Nodemailer (SMTP interface for emailing generated invoices).

3.4 Architecture Blueprint

Inventrix is built on the Model-View-Controller (MVC) architecture on the backend coupled with a responsive Single Page Application (SPA) on the frontend. Client dashboards establish direct websocket streams back to the server. API controllers process requests securely by validating incoming JWT signatures and cross-checking user permissions against custom role databases before altering records.

4. Project Structure Walkthrough

The codebase is clean and modular. The folder structures and files are arranged as follows:

```
inventrix/
├── backend/                  # Server-side environment
│   ├── config/               # Database and configuration files
│   ├── controllers/          # Business logic handlers (auth, sales, invoices, etc.)
│   ├── middleware/           # Security, authentication, and RBAC controllers
│   ├── models/               # MongoDB/Mongoose database schema definitions
│   ├── routes/               # API endpoint configurations
│   ├── utils/                # Helper files (SMTP configuration, default role seeders)
│   ├── index.js              # Server entry point (Express, Socket.io, and DB
connection)
│   └── .env                  # Port, Database URI, and SMTP credentials
├── src/                      # Frontend environment
│   ├── components/           # Reusable UI React components
│   │   ├── common/           # Shared modules (Custom DatePicker, Toast alerts)
│   │   └── dashboard/        # Dashboard layout components (Owner, Employee, Supplier)
│   ├── pages/                # Page layouts (Login, Customer Portal, Dashboards)
│   ├── styles/               # Styling sheets (.css) for themed elements
│   ├── App.jsx               # Application routes controller
│   └── main.jsx              # DOM rendering controller
└── package.json              # Directives and execution script configurations
```

5. Core Platform Features

5.1 Handheld Barcode Scanning

Captures UPC/EAN scans from active scanner inputs and adds the matching database item to the POS cart.

• **Why it is important:** Speeds up the checkout process, removes manual search errors, and links client checkout carts directly to warehouse inventories.

• **How it works internally:**
1. Cashier opens the checkout dashboard and establishes a Socket.io listener for 'product-scanned' events.
2. Cashier scans a product's barcode with a handheld hardware device.
3. The hardware sends the barcode payload to the server's Socket.io scanner channel.
4. The backend queries MongoDB for a matching product in the store's inventory.
5. The server emits the product details back to the cashier's socket connection, which updates the cart state instantly.

5.2 Multi-Store Financial Management

Provides store owners with real-time visual cards rendering total sales, employee headcounts, total products, and financial totals (Revenue, Cost, and Profit calculations).

• **Why it is important:** Allows store owners managing multiple branches to monitor the profitability and inventory levels of each location from a single dashboard.

• **How it works internally:**
1. The Owner Dashboard mounts and requests stats from the `/api/sales/analytics/All` or a specific store ID.
2. The server queries the database, aggregates cost prices and selling prices for all items in completed sales.
3. It computes the net metrics: Revenue (Sum of sale totals), Cost (Sum of purchase costs), and Profit (Revenue minus Cost).
4. The values are returned and rendered in visual KPI cards on the dashboard screen.

5.3 Timezone-Aware Analytics (IST)

Aggregates historical sales transactions by date/time, converting stored UTC timestamps to Indian Standard Time (IST) before grouping.

• **Why it is important:** Ensures that sales reports reflect actual local shopping behavior in India, preventing UTC time-shifts from placing sales into the wrong hours.

• **How it works internally:**
1. The dashboard charts request analytical data from the server.
2. The API uses MongoDB's aggregation framework to parse transaction records.

3. Inside the pipeline, the `$hour` operator is configured with the timezone parameter set to 'Asia/Kolkata'.
4. MongoDB performs the timezone conversion offset (+5:30) dynamically prior to aggregation.
5. The results are grouped by the converted local hour and returned to the client for rendering.

5.4 Custom Role-Based Access Controls (RBAC)

Enables owners to define custom roles (e.g. Manager, cashier) and specify granular read/write permissions for employee accounts.

• **Why it is important:** Prevents unauthorized employees from editing product listings, issuing manual stock updates, or reading financial reports.

• **How it works internally:**
1. The owner creates a custom role profile and assigns system permissions (e.g., 'edit_inventory': false).
2. When an employee logs in, their JWT token is generated containing their custom role ID.
3. During API requests, the authorization middleware retrieves the role permissions from the database.
4. The middleware validates whether the route's required permission matches the user's role.
5. If authorization fails, the API returns a 403 Forbidden status, denying access.

5.5 B2B Supply Procurement Integrations

Enables owners to create purchase orders directly to suppliers. Suppliers can review and accept orders, setting delivery timelines.

• **Why it is important:** Automates the restocking workflow, locks unit prices to default wholesale costs, and keeps logs transparent between stores and suppliers.

• **How it works internally:**
1. The owner creates a purchase order, selecting items from the supplier's wholesale catalog.
2. The system locks the unit price to the supplier's wholesale catalog rate and saves the PO status as 'pending'.
3. The supplier receives a real-time notification on their dashboard, goes to the PO list, and selects the order.
4. The supplier inputs the Expected Delivery Date and optional notes, changing the status to 'accepted'.
5. The store owner's dashboard automatically refreshes, displaying the updated PO details and notes.

5.6 Calibri-Rendered PDF Invoices & Nodemailer System

Generates a formal transaction invoice PDF containing the Indian Rupee symbol (₹) and Indian formatted dates, and emails it to the customer.

• **Why it is important:** Ensures tax-compliant billing layouts, formats currencies locally, and automates digital receipt delivery directly to the buyer's inbox.

• **How it works internally:**
1. Cashier completes checkout and provides the customer's email address.
2. The server creates an invoice document, fetching the local Calibri font to support Indian character sets.
3. PDFKit compiles the invoice layout on disk with localized date strings and the Unicode currency symbol ('₹').
4. Nodemailer establishes a secure SMTP socket connection and transmits the PDF as an email attachment.
5. The customer receives their receipt, and the transaction is closed.

5.7 Real-Time Alert System

Pushes instant visual notifications (out of stock alerts, PO status updates) to relevant users, refreshing their screens immediately.

• **Why it is important:** Prevents cashiers from trying to sell out-of-stock products and alerts owners to checkouts or deliveries as they occur.

• **How it works internally:**
1. A cashier completes a checkout that reduces a product's stock count to zero.
2. The database trigger initiates a socket message containing the alert description.
3. The WebSocket server pushes a 'newNotification' event to the user's unique socket ID.
4. The client dashboard UI increments its local refresh key.
5. The page automatically re-fetches background API metrics, displaying the alert and updating stats in real-time.

6. Core Codebase Functionality & Explanations

6.1 Peak Hour Aggregation (Timezone Pipelines)

This function is located in backend/controllers/salesController.js. It queries historical sales and groups them into hourly buckets after converting UTC timestamps into Asia/Kolkata (IST) time.

```
// Excerpt from backend/controllers/salesController.js
// 2. Hourly Sales (Peak Hours)
const hourlySales = await Sale.aggregate([
    { $match: matchQuery },
    {
        $group: {
            _id: { $hour: { date: "$date", timezone: "Asia/Kolkata" } },
            count: { $sum: 1 },
            revenue: { $sum: "$totalAmount" }
        }
    },
    { $sort: { _id: 1 } }
]);
```

• **Line-by-Line Explanation:**
Line 3: Initiates the aggregation framework on the Sale model collection.
Line 4: Match Stage: Filters transactions based on criteria like status 'completed' and selected store ID.
Line 5: Group Stage: Groups matching transactions. The group key _id is set by extracting the hour portion of the sale's 'date' attribute using the timezone conversion property. The server translates UTC values to Indian Standard Time (+5:30) dynamically.
Line 8-9: Aggregates total sales count (count) and total amount (revenue) per hour bucket.
Line 12: Sort Stage: Orders the aggregated hours chronologically from 0 to 23.

• **Inputs & Outputs:** Takes a query object filter (storeId). Outputs an array of hourly metrics. Edge Case: Handles missing records by filling empty hours with default zero values in subsequent controller steps.
• **Critical Importance:** Provides store owners with accurate data regarding peak sales hours, enabling optimized staff scheduling and inventory preparation.

6.2 Dynamic Unicode PDF Invoice Creator

This code resides in backend/controllers/invoiceController.js. It compiles PDF documents using a registered Calibri font format to support rendering unicode characters such as '₹'.

```
// Excerpt from backend/controllers/invoiceController.js
const FONT_REGULAR = "C:\\Windows\\Fonts\\calibri.ttf";
const FONT_BOLD    = "C:\\Windows\\Fonts\\calibrib.ttf";

const formatCurrency = (value) => {
    return `\u20B9 ${Number(value).toFixed(2)}`;
};
```

```
const createPdfAndSend = async (invoiceRecord) => {
    const doc = new PDFDocument({ margin: 50 });
    doc.registerFont("calibri",      FONT_REGULAR);
    doc.registerFont("calibri-bold", FONT_BOLD);

    const stream = fs.createWriteStream(filePath);
    doc.pipe(stream);

    doc.fillColor("#111827")
       .font("calibri-bold")
       .fontSize(9)
       .text(formatCurrency(subtotal), 460, y + 2, { width: 85, align: "right" });
    // ...
};
```

• **Line-by-Line Explanation:**
Line 2-3: References the system-level Calibri regular and bold font files.
Line 5: formatCurrency: Returns values prefixed with the unicode Rupee symbol character (₹).
Line 9: Creates a new PDFKit document instance.
Line 10-11: Registers the fonts under the aliases 'calibri' and 'calibri-bold'.
Line 13-14: Redirects the generated PDF output stream to a file on disk.
Line 16-19: Styles the font and prints the formatted rupee amount at precise layout coordinates.

• **Inputs & Outputs:** Inputs include the sale record and target customer email address. Outputs a PDF file on disk and dispatches a Nodemailer SMTP transmission. Edge Case: Falls back to Segoe UI or Arial if Calibri is missing.
• **Critical Importance:** Ensures professional, localized receipts containing accurate currency representation, automating document delivery on completion of purchase.

6.3 JWT Security Validation Middleware

This security middleware sits in backend/middleware/auth.js, validating user sessions and loading profiles based on the verified payload type.

```
// Excerpt from backend/middleware/auth.js
const authenticate = async (req, res, next) => {
    try {
        let token;
        if (req.headers.authorization && req.headers.authorization.startsWith('Bearer')) {
            token = req.headers.authorization.split(' ')[1];
        }
        if (!token) return res.status(401).json({ success: false, message: 'Not
authorized' });

        const decoded = jwt.verify(token, process.env.JWT_SECRET);
        let user;
        if (decoded.userType === 'employee') {
            user = await Models.Employee.findById(decoded.id).populate('role').select('-
password');
        } else if (decoded.userType === 'store_owner') {
```

```
            user = await Models.StoreOwner.findById(decoded.id).select('-password');
        }
        if (!user) return res.status(401).json({ success: false, message: 'User not found'
});

        req.user = user;
        req.userType = decoded.userType;
        next();
    } catch (error) {
        return res.status(401).json({ success: false, message: 'Auth failed' });
    }
};
```

• **Line-by-Line Explanation:**
Line 4-6: Extracts the bearer token payload from the Authorization request header.
Line 7: Rejects the request with a 401 Unauthorized status if no token is found.
Line 9: Decodes the token using the secret key JWT_SECRET.
Line 11-15: Queries the corresponding database collection (Employee or StoreOwner) based on the userType.
Line 16: Denies access if the user record does not exist.
Line 18-20: Appends the authenticated profile data to the request object and triggers the next middleware.

• **Inputs & Outputs:** Takes HTTP headers. Appends the validated `req.user` profile to downstream endpoints. Edge Case: Instantly flags deactivated employee status.
• **Critical Importance:** Secures internal database operations, verifying roles and permissions to block unauthorized access.

6.4 Persistent Socket.io Listener Hub

Located in src/pages/dashboard/OwnerDashboard.jsx. Moves the Socket.io instance creation outside of component hooks to prevent reconnect loops, and manages component subscriptions cleanly on mount.

```
// Excerpt from src/pages/dashboard/OwnerDashboard.jsx
import { SOCKET_URL } from "../../config";
import { io } from "socket.io-client";

// Persistent socket instance at module level (not inside the component)
const socket = io(SOCKET_URL);

const OwnerDashboard = () => {
    // ...
    // Join personal user room as soon as user._id is known
    useEffect(() => {
        if (user?._id) {
            socket.emit("join-user", user._id);
        }
    }, [user?._id]);

    // Register store and notification socket listeners once on mount
    useEffect(() => {
```

```
        const handleNewNotification = (notification) => {
            setNotifications((prev) => [notification, ...prev]);
            setRefreshKey((prev) => prev + 1);
        };
        socket.on("newNotification", handleNewNotification);
        return () => {
            socket.off("newNotification", handleNewNotification);
        };
    }, []);
};
```

• **Line-by-Line Explanation:**
Line 5: Establishes a single socket connection to SOCKET_URL on script initialization.
Line 10-14: Sends a join-user request to register the authenticated user in their own room.
Line 17: Sets up the socket event listener inside a mount effect with an empty dependency array, registering it exactly once.
Line 18-21: When newNotification is received, it updates the notification list and increments the refresh key, triggering a data update.
Line 24: Removes the event listener on component unmount to prevent duplicate registrations and memory leaks.

• **Inputs & Outputs:** Listens for incoming Socket.io notifications. Outputs UI updates and increments the refreshKey. Edge Case: Handles network drops by auto-reconnecting.
• **Critical Importance:** Provides real-time inventory updates and alert pushes, keeping the user interface current without browser reloads.

6.5 Responsive Flexbox Layout (CSS Stylesheet)

Located in src/styles/SupplierDashboard.css. It aligns form elements side by side on desktop and tablet viewports while stacking them on mobile screens.

```
/* Excerpt from src/styles/SupplierDashboard.css */
.po-response-row {
    display: flex;
    flex-direction: column;
    gap: 1rem;
    margin-bottom: 0.8rem;
}

@media (min-width: 600px) {
    .po-response-row.side-by-side {
        flex-direction: row;
        align-items: flex-start;
        gap: 1.5rem;
    }
    .po-response-row.side-by-side .date-field {
        flex: 1;
        margin-bottom: 0;
    }
    .po-response-row.side-by-side .text-field {
        flex: 2;
```

```
        margin-bottom: 0;
    }
}
```

• **Line-by-Line Explanation:**
Line 2-7: Configures a vertical layout for mobile devices, using column flex direction and spacing.
Line 9: Media Query: Applies layout overrides for viewport widths of 600px and above.
Line 10-14: Switches to a row layout, aligning items to the top and setting a horizontal gap.
Line 15-18: Allocates 1/3 of the container space to the DatePicker field.
Line 19-22: Allocates 2/3 of the container space to the textarea notes field.

• **Inputs & Outputs:** Inputs include viewport dimensions. Outputs custom responsive styling adjustments. Edge Case: Adapts cleanly to dynamic browser resizing.
• **Critical Importance:** Improves form readability and usability on the Supplier portal, optimizing screen space.

7. End-to-End System Data Flow

To illustrate how information flows through the system, here is the step-by-step lifecycle of a checkout transaction:

- **1. Item Scanning:** A cashier scans a barcode. The scan emits a Socket.io event that retrieves the matching product from MongoDB and adds it to the checkout cart state.
- **2. Cart Validation:** The POS calculates the subtotal, taxes, discounts, and grand total. When the cashier clicks 'Checkout', the cart details are sent to `/api/sales/` via a POST request.
- **3. Inventory Deductions:** The backend verifies stock levels. If the checkout quantity is valid, the stock count is decremented in MongoDB. If stock falls to zero, a low stock notification is triggered.
- **4. Real-time Pushes:** The server emits a low stock event (`newNotification`) to the store room. Active dashboards receive this, increment their refresh key, and refresh their data automatically.
- **5. Billing PDF Compilation:** The server creates an invoice PDF, retrieving Calibri fonts to render the Indian Rupee symbol (₹) and Indian formatted dates (DD/MM/YYYY) correctly.
- **6. SMTP Transmission:** Nodemailer connects to the SMTP server and emails the generated PDF invoice to the customer.

8. Setup & Installation Manual

8.1 Prerequisites

Ensure your system meets the following environment dependencies before installation:

- • **Node.js:** LTS version 20 or higher installed.
- • **Database:** MongoDB local server or MongoDB Atlas cluster connection URI.
- • **Package Manager:** npm (v10 or higher) installed.

8.2 Installation Steps

```
# Clone the repository
git clone https://github.com/your-repo/inventrix.git
cd inventrix

# Install root dependencies
npm install

# Install backend dependencies
cd backend
npm install
cd ..

# Run the development environment (starts frontend and backend concurrently)
npm run both
```

8.3 Environment Configurations (`backend/.env`)

Create a .env file in the backend folder and configure the following variables:

```
PORT=5000
MONGO_URI=mongodb+srv://<user>:<password>@cluster.mongodb.net/inventrix?retryWrites=true&w
=majority
JWT_SECRET=your_signing_secret_key
JWT_REFRESH_SECRET=your_refresh_signing_secret_key
SOCKET_URL=http://localhost:5000

# Nodemailer Configuration (SMTP Gateway)
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=587
EMAIL_SECURE=false
EMAIL_USER=your-smtp-authorized-email-sender@gmail.com
EMAIL_PASS=your-smtp-authorized-passkey
EMAIL_FROM="Inventrix Store Invoicing" <your-smtp-authorized-email-sender@gmail.com>
```

9. Future Scope & Enhancements

The following improvements are planned for future updates to the platform:

- • **Offline Database Caching:** Integrating Service Workers and IndexedDB to allow caching transactions and syncing checkout logs when internet connectivity is restored.
- • **Intelligent Forecasting:** Using machine learning on sales history to forecast stock demands and automate B2B purchase orders.
- • **Batch Purchase Order Processing:** Allowing owners to group purchase orders for multiple stores and suppliers into a single transaction.

10. Architectural Conclusion

Inventrix provides retail store owners with a unified, real-time platform to coordinate transactions, staff operations, and wholesale procurement. By decoupling core frontend layouts and leveraging persistent WebSockets, the platform reduces latency and automates data updates. Secure endpoints, custom role-based permissions, and localized invoice generation provide a reliable, premium tool for retail business management.